
PneumoCheck

Détection de Pneumonie par Analyse de Radiographies Thoraciques

Cahier des Charges — Mini Projet

Module : Développement Mobile Cross-Plateforme - Flutter

Étudiant : Karoui Mohamed Aziz

Groupe : Groupe 3 — 4^{ème} année Data Science

Année : 2025 / 2026

Table des matières

1	Présentation du Projet	2
1.1	Contexte	2
1.2	Objectif principal	2
1.3	Lien du Dataset	2
2	Stack Technique	2
3	Fonctionnalités de l'Application	3
3.1	Module Authentification	3
3.2	Module Analyse IA	3
3.3	Module Historique	3
3.4	Module Statistiques / Dashboard	3
3.5	Fonctionnalités Transversales	3
4	Architecture Flutter	4
4.1	Structure des dossiers	4
4.2	Gestion de l'état (Riverpod)	4
5	Backend FastAPI	5
5.1	Endpoints principaux	5
5.2	Pipeline de prédiction	5
6	Modèle d'Intelligence Artificielle	5
6.1	Architecture du modèle	5
6.2	Grad-CAM	5
7	Persistance des Données	6

Présentation du Projet

Contexte

La pneumonie est l'une des premières causes de mortalité infantile dans le monde. Son diagnostic repose traditionnellement sur l'analyse manuelle de radiographies thoraciques par un radiologue, un processus long et coûteux.

PneumoCheck est une application mobile Flutter qui exploite un modèle de deep learning (CNN) pour analyser automatiquement des clichés radiographiques et détecter des signes de pneumonie, offrant ainsi une aide au diagnostic rapide et accessible.

Objectif principal

Concevoir une application mobile Flutter complète, intégrant un pipeline d'IA (CNN + Grad-CAM) exposé via une API FastAPI, permettant à un utilisateur d'uploader une radiographie thoracique et d'obtenir en quelques secondes un diagnostic assisté avec visualisation des zones suspectes.

Lien du Dataset

Dataset Kaggle — Chest X-Ray Images (Pneumonia)

Auteur : Paul Timothy Mooney

<https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

Nombre d'images : 5 863 radiographies

Classes : NORMAL / PNEUMONIA

Split : Train / Val / Test — déjà fourni

Licence : CC BY 4.0 (utilisation libre)

Stack Technique

Couche	Technologie
Frontend Mobile	Flutter (Dart) + Riverpod
Backend API	Python — FastAPI
Modèle IA	TensorFlow / Keras — EfficientNetB0 fine-tuné
Visualisation IA	Grad-CAM (heatmap des zones suspectes)
Graphiques	fl_chart (Flutter)
Persistance	flutter_secure_storage (local token) + PostgreSQL (serveur)
Authentification	JWT (JSON Web Token)
Conteneurisation	Docker (backend FastAPI)

TABLE 1 – Stack technique du projet PneumoCheck

Fonctionnalités de l'Application

Module Authentification

- Inscription via formulaire (nom, email, mot de passe)
- Connexion avec génération de token JWT
- Persistance de session (token stocké dans flutter_secure_storage)
- Déconnexion et invalidation du token
- Gestion des erreurs (champ vide, email invalide, mauvais identifiants)

Module Analyse IA

- Upload d'une radiographie thoracique depuis la galerie ou la caméra
- Envoi de l'image au backend FastAPI via requête multipart
- Prédiction par le modèle CNN (EfficientNetB0) : classe + score de confiance
- Génération de la **heatmap Grad-CAM** superposée sur la radio
- Affichage du résultat : badge coloré **NORMAL** (vert) / **PNEUMONIE** (rouge)
- Pourcentage de confiance affiché avec une barre de progression animée

Module Historique

- Sauvegarde automatique de chaque analyse (image + résultat + date)
- Consultation de l'historique en mode liste
- Suppression d'une entrée de l'historique
- Disponibilité offline (optionnel : cache local si implémenté — ex. sqflite)
- Synchronisation avec le serveur si connexion disponible

Module Statistiques / Dashboard

- Nombre total d'analyses effectuées
- Répartition NORMAL vs PNEUMONIE (graphique camembert — fl_chart)
- Courbe d'évolution des analyses sur les 30 derniers jours (graphique linéaire)
- Score de confiance moyen par classe

Fonctionnalités Transversales

- Gestion des erreurs réseau : timeout API, absence de connexion internet
- Indicateurs de chargement (loading spinners) pendant les appels API
- Messages Snackbar pour les succès et les erreurs
- Interface responsive (téléphone + tablette)

Architecture Flutter

Structure des dossiers

```
lib/
|-- core/
|   |-- constants/      # couleurs, URLs, chaines
|   |-- errors/         # exceptions personnalisées
|   |-- network/        # client HTTP (Dio)
|   +-- utils/          # fonctions utilitaires
|-- features/
|   |-- auth/
|   |   |-- data/       # AuthRepository, AuthService
|   |   |-- domain/     # modèles User, AuthState
|   |   +-- presentation/ # LoginPage, RegisterPage
|   |-- scan/
|   |   |-- data/       # ScanRepository, API calls
|   |   |-- domain/     # modèles ScanResult
|   |   +-- presentation/ # ScanPage, ResultPage
|   |-- history/
|   |   |-- data/       # HistoryRepository (local cache / server sync)
|   |   |-- domain/     # modèle HistoryEntry
|   |   +-- presentation/ # HistoryPage
|   +-- dashboard/
|       +-- presentation/ # DashboardPage (fl_chart)
-- models/                # modèles de données (pas de Hive)
-- widgets/               # widgets réutilisables
-- main.dart
```

Gestion de l'état (Riverpod)

Provider	Rôle
authNotifierProvider	Gère l'état de connexion et le token JWT
scanNotifierProvider	Gère l'image uploadée, la prédiction et la heatmap
historyNotifierProvider	Gère la liste des analyses sauvegardées
dashboardProvider	Agrège les statistiques pour les graphiques
connectivityProvider	Surveille l'état de la connexion réseau

TABLE 2 – Providers Riverpod du projet

Backend FastAPI

Endpoints principaux

Méthode	Route	Description
POST	/auth/register	Inscription utilisateur
POST	/auth/login	Connexion + retour token JWT
POST	/scan/predict	Upload radio → prédiction + Grad-CAM
GET	/history/{user}	Récupère l'historique d'un utilisateur
DELETE	/history/{id}	Supprime une analyse
GET	/stats/{user}	Statistiques agrégées pour le dashboard

TABLE 3 – Endpoints de l'API FastAPI

Pipeline de prédiction

1. Réception de l'image (multipart/form-data)
2. Prétraitement : redimensionnement à 224×224 , normalisation
3. Inférence via EfficientNetB0 fine-tuné
4. Calcul Grad-CAM : carte de chaleur sur la dernière couche convolutive
5. Superposition heatmap + image originale
6. Retour JSON : `label`, `confidence`, `heatmap_base64`

Modèle d'Intelligence Artificielle

Architecture du modèle

- **Base** : EfficientNetB0 pré-entraîné sur ImageNet
- **Fine-tuning** : couches de classification remplacées (Dense 256 + Dropout + Dense 1 sigmoid)
- **Entrée** : image 224×224 px, 3 canaux RGB
- **Sortie** : probabilité $p \in [0, 1]$ — si $p > 0.5$: PNEUMONIE, sinon : NORMAL
- **Performances attendues** : accuracy $\geq 92\%$ sur le jeu de test Kaggle

Grad-CAM

La technique Grad-CAM (*Gradient-weighted Class Activation Mapping*) génère une carte de chaleur indiquant les régions de la radiographie ayant le plus influencé la décision du modèle. Cela permet de visualiser concrètement les zones suspectes et d'augmenter la confiance dans le diagnostic.

Persistence des Données

Donnée	Stockage	Justification
Token JWT	flutter_secure_storage (local)	Persistence de session sécurisée localement
Historique scans	PostgreSQL (+ cache local optionnel)	Consultation et synchronisation côté serveur
Images analysées	Stockées côté serveur (base64/URL)	Affichage et archivage centralisés
Statistiques	PostgreSQL	Agrégation côté serveur

TABLE 4 – Stratégie de persistance des données